

WORKSHOP SERIES

SESSION 01

Deep Dive AI Workshop Series

Foundations of Large Language Models

Session 1 · March 2026

Aryan Mishra

Robotics Team Lead · 4+ Years in AI Systems & LLM Development

Architect and deploy production-scale LLM solutions on Azure and AWS. Lead cross-functional teams from POC through production, building multi-agent AI systems, RAG-based solutions, and conversational AI applications across banking, hospitality, and IT services.

Core Expertise

Multi-Agent AI Systems

RAG Architecture

LLM Pipelines

Conversational AI

Cloud-Native GenAI App

Physical AI / Robotics



Connect With Me

LinkedIn

linkedin.com/in/mishra-aryan

GitHub

github.com/aryanmishra98-08

Medium

medium.com/@aryanmishra98.08

Email

aryanmishra98.08@gmail.com

Topics Covered in This Session

01

Foundations of LLMs

What LLMs are, how they learn from large-scale text data, and the role of neural networks.

02

Core Architecture

Transformer architecture and how LLMs generate text by predicting the next token.

03

Model Training & Improvement

Fine-tuning techniques and reinforcement learning with human feedback (RLHF).

04

Applications & Capabilities

Common real-world uses of LLMs across various domains.

05

Limitations & Reasoning

Cognitive limits of LLMs and how reasoning methods and external tools help mitigate them.

06

Challenges & Risks

Technical limitations, security challenges, and ethical considerations.

07

Emerging AI Landscape

Multimodal AI, autonomous agents, and voice-based systems.

08

Current AI Landscape (2026)

Overview of major models and the current AI ecosystem.

09

Hands-on Development

Building an LLM wrapper from scratch.

Foundations of LLMs

01

What is a Language Model?

A system designed to predict the next word, phrase, or character in a text sequence based on preceding context — similar to autocomplete, but using patterns learned from vast text data to generate coherent, relevant outputs.

02

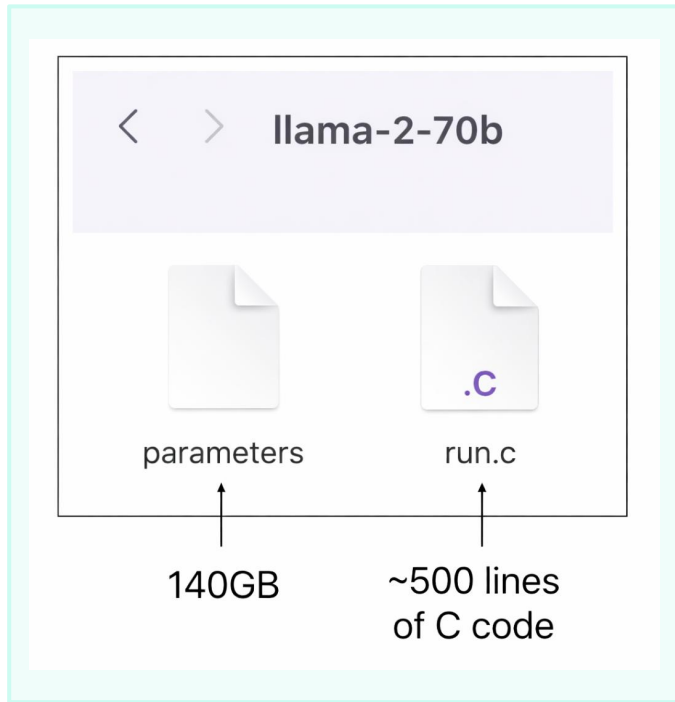
When Does It Become “Large”?

When trained on extremely large datasets with billions of parameters. This scale enables broader context understanding, complex pattern capture, and emerging capabilities like reasoning, summarisation, and conversation.

03

How Does an LLM Look Internally?

Two key components: a Parameter File (trained weights storing learned knowledge) and Runtime Code (software that loads weights and performs inference to generate outputs from input prompts).



Foundations of LLMs

Large Language Models are a specialised subset within a broader AI hierarchy, focused on understanding and generating human language at scale.

Artificial Intelligence (AI)

Broad domain of machines performing intelligent tasks.

Natural Language Processing (NLP)

Subfield focused on understanding and generating human language.

Language Models (LMs)

Models that predict or generate text.

Large Language Models (LLMs)

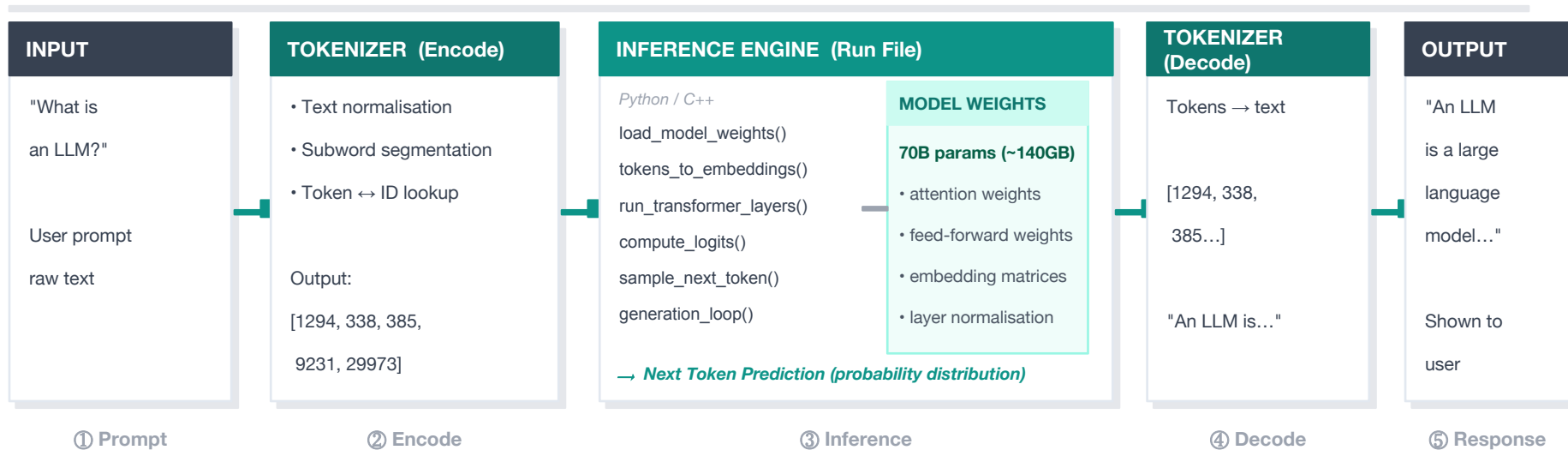
Very large language models trained on massive datasets using transformer architectures.

What NLP Includes

- 1 Text classification
- 2 Sentiment analysis
- 3 Named Entity Recognition (NER)
- 4 Machine translation
- 5 Speech-to-text
- 6 Question answering
- 7 Chatbots

These tasks can be implemented using rule-based systems, statistical models, or deep learning models like LLMs.

How an LLM Processes Your Input



Key Takeaway

The parameter file (model weights) is static — it never changes at runtime. Only the run file executes logic. Inference is purely mathematical: tokens in, probability distribution out, next token sampled.

SESSION 01 · FOUNDATIONS

How Are LLMs Built?

Pre-Training

Building the Foundation of an LLM

The pre-training stage establishes the foundational knowledge of an LLM. The model learns patterns, grammar, facts, and relationships from massive amounts of text data across three key steps.

01

Data Acquisition

Collecting large volumes of text data from diverse sources — websites, books, articles, forums, and public datasets.

02

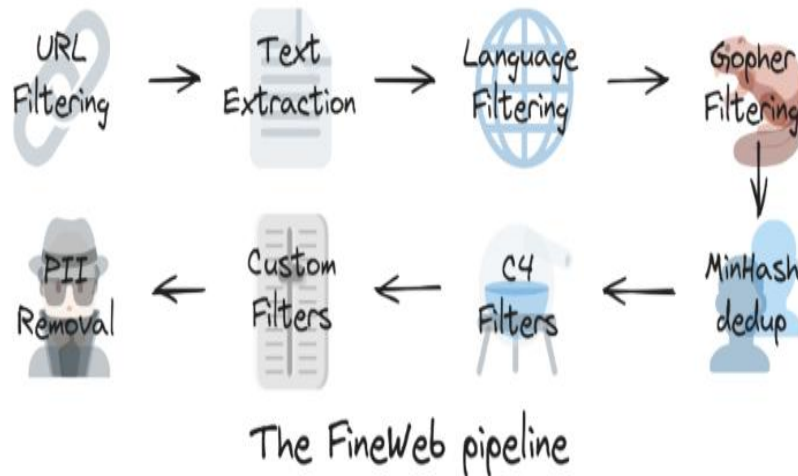
Data Filtering & Quality Control

Cleaning and refining collected data to remove noise, low-quality text, and sensitive information.

03

Tokenization

Converting raw text into smaller units (tokens) that the model can process and learn from.



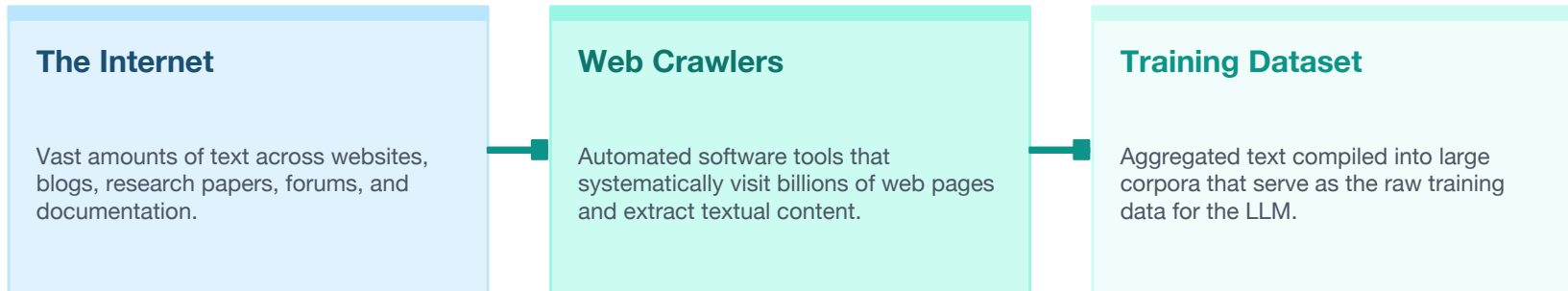
Key Point

The quality of pre-training data directly determines the quality of the model's knowledge and reasoning ability.

Data Acquisition

Collecting Large-Scale Training Data

Large Language Models are trained on massive datasets gathered from the internet and other publicly available sources.



→ Results in a massive raw dataset that will later undergo cleaning and filtering.

EXAMPLE DATASET

FineWeb Dataset

Available on Hugging Face

A curated large-scale dataset derived from web content and commonly used for training modern language models. It demonstrates what a real-world training corpus looks like after acquisition.

Data Filtering and Quality Control

Cleaning and Preparing the Dataset

Once raw data is collected, it must be filtered to ensure quality, safety, and usability for model training.

01 URL & Domain Filtering

Removing low-quality or unreliable sources before any text is even extracted.

02 Text Extraction Filtering

Ensuring only meaningful textual content is retained; stripping HTML, scripts, and boilerplate.

03 Language Detection Filtering

Keeping only the target language(s), discarding non-relevant content.

04 Duplicate Removal

Eliminating repeated content to improve dataset diversity and prevent memorisation.

05 Privacy & Sensitive Content

Removing personal data, harmful content, and sensitive information before training.

Result

A clean, high-quality dataset that can be safely and effectively used for training the language model.

Tokenization

Before a model can process language, text must be converted into numbers. This starts with representing text as a flat, one-dimensional stream of characters.

STEP 1 → 1D Text Sequence

in the early years of computing scientists began exploring the possibility that machines might one day process human language. at first the approaches were simple and rule based...

STEP 2 → UTF-8 Binary Encoding

01101001 01101110 00100000 01101000 01101000 01100101 00100000
01100101 01100001 01110010 01101100 01111001...

STEP 3 → Byte Values

[105, 110, 32, 116, 104, 101, 32, 101, 97, 114, 108, 121]
→ each character becomes an integer byte value

Char → UTF-8 Binary → Byte

i	01101001	105
n	01101110	110
(space)	00100000	32
t	01110100	116
h	01101000	104
e	01100101	101
(space)	00100000	32
e	01100101	101
a	01100001	97
r	01110010	114
l	01101100	108
y	01111001	121

Each character maps to a unique binary pattern (UTF-8), read as an integer byte value by the model.

Byte Pair Encoding (BPE)

BPE iteratively merges the most frequent character pairs into subword tokens, building a vocabulary from the ground up.

STEP 1 · Starting Text

in the early years

Fragment of raw training text

STEP 2 · Initial Symbols

in_the_early_years

_ = space. Split into characters/bytes

STEP 3 · Count Frequent Pairs

(t,h) (h,e) (e,r) (e,a) (l,y)
→ Most frequent: th, he, er, ea, ly

Pairs counted across full dataset

STEP 4 · First Merge: th

th → th
in_the_early_years

Most frequent pair merged

STEP 5 · Second Merge: the

the → the
in_the_early_years

Continue with next frequent pair

STEP 6 · Final Tokens + IDs

in | the | ear | ly | years
→ [452, 12, 981, 77, 2045]

Each token maps to a numeric ID

Full Tokenization Pipeline

How raw text travels through the tokenization process before reaching the transformer.

Raw Text

Natural language input

UTF-8 Bytes

Each character → binary → byte integer

Initial Symbols

Characters / bytes split into base units

BPE Merge Operations

Frequent pairs iteratively merged

Subword Tokens

in | the | ear | ly | years

Token IDs

[452, 12, 981, 77, 2045]

Example Walkthrough

Raw Text

"in the early years"

UTF-8 Bytes

[105, 110, 32, 116, 104, 101, 32...]

Characters

i n _ t h e _ e a r l y

After BPE

in | the | ear | ly | years

Token IDs

[452, 12, 981, 77, 2045]

Next step

Token IDs → embedding vectors → transformer

Neural Network Training

Modern Large Language Models rely on two core technologies: neural network training and the transformer architecture. Together, they transform random parameters into models capable of understanding and generating human language.

01 Billions of Random Parameters

At the start of training, the model contains billions of tiny adjustable values (parameters) initialised with random values — no knowledge of language.

02 Next-Token Prediction Loop

The model predicts the next token, compares it with the actual token using a loss function, and adjusts parameters to reduce error — repeated billions of times.

03 Predictive Typing Analogy

Like predictive typing on a smartphone — early predictions seem random, but the system steadily improves as it learns patterns from data.

Predict Next Token

Compare with Actual

Compute Loss

Adjust Parameters

Scaling Large Language Models

As models grow larger, their capabilities improve significantly — but so do their training requirements.

175B

Parameters in GPT-3

Modern LLMs contain billions or hundreds of billions of parameters

01

High-Performance GPUs

AI accelerators such as NVIDIA H100, capable of massive parallel computation.

02

Massive Compute

Extremely large numbers of mathematical operations across enormous datasets.

03

Distributed Training

Workloads split across many machines to handle the computational demand at scale.

Key Takeaway

Scale is what differentiates large language models from earlier approaches. The combination of massive parameter counts, enormous datasets, and distributed GPU infrastructure enables the emergent capabilities (reasoning, summarisation, conversation) that make modern LLMs so powerful.

From Training to Real-World Use

Once training is completed, the model transitions from learning patterns to applying them in real-world tasks. This stage is known as inference.

Training

Learning patterns from data



Inference

Applying knowledge to tasks

What is Inference?

The stage where the model uses learned knowledge to perform tasks — this is when it becomes a practical tool.

01 Text Generation

Creating coherent text from prompts

02 Summarisation

Condensing long content into key points

03 Translation

Converting text between languages

04 Question Answering

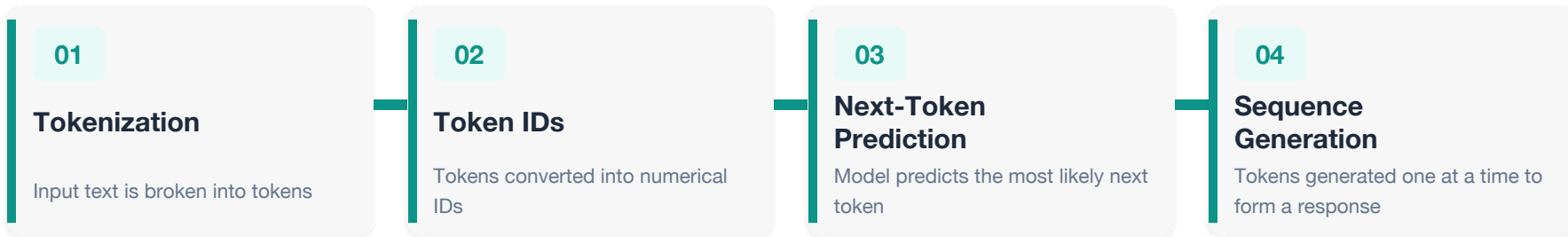
Providing answers from context

05 Content Creation

Generating creative and professional content

How LLM Inference Works

Inference is the process by which a trained Large Language Model generates text from a user prompt.



Example

Prompt: "The sun is"

Possible predictions:

bright

shining

hot

rising

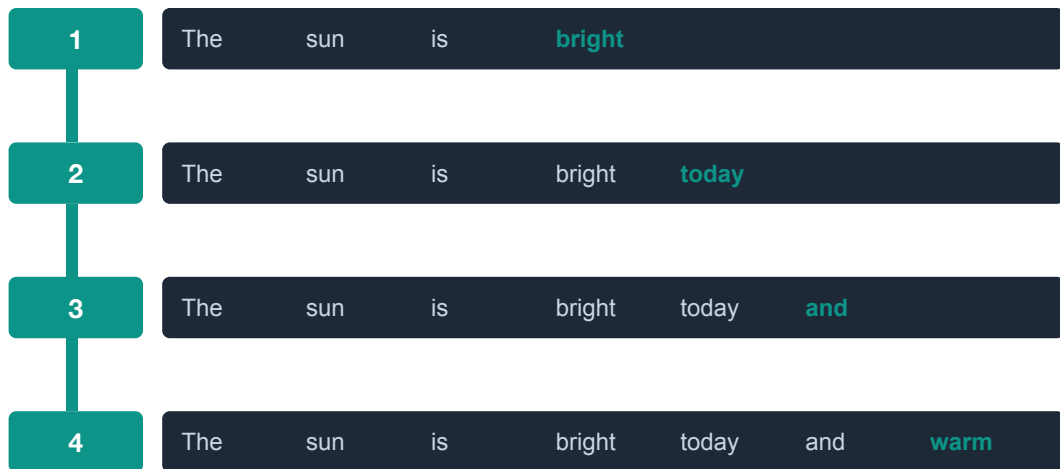
How it works

The model assigns probability scores to all tokens in its vocabulary, then selects a token and continues building the sentence.

Autoregressive Text Generation

LLMs generate text using an autoregressive process — the model produces one token at a time, using previously generated tokens as context.

Step-by-Step Generation



How It Works

Each token prediction depends on the entire sequence generated so far, allowing the model to maintain contextual coherence throughout the response.

Token 1 context: prompt only

Token 2 context: prompt + token 1

Token 3 context: prompt + tokens 1–2

Token N context: all previous tokens

Key Point

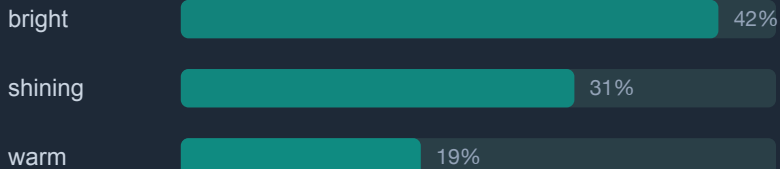
"Autoregressive" means each output token becomes input for the next prediction — the model builds text sequentially, never in parallel.

Nucleus Sampling & Temperature

Top-P (Nucleus) Sampling

Instead of a fixed number of tokens, the model chooses from the smallest set whose cumulative probability exceeds a threshold (e.g., 90%).

Candidate Set ($p > 90\%$)



→ A token is randomly selected from this probability mass

Temperature Control

Temperature adjusts the randomness of token selection during generation.

Low Temperature

Deterministic, precise output

The sun is bright.

High Temperature

Creative, diverse output

The sun is blazing across the sky.

0.0



2.0

Focused → Creative

Supervised Fine-Tuning (SFT)

Supervised Fine-Tuning transforms a pretrained language model into a conversational assistant.

During pretraining, the model learns to predict the next word based on statistical patterns. While this enables strong language understanding, it does not automatically produce structured human conversations.

SFT trains the model to generate responses that are:

Helpful

Truthful

Harmless

Pretrained Model

Next-word predictor

Conversational Assistant

Example

User: "Can you recommend landmarks in Paris?"

Goal: Clear, informative, conversational

Assistant: "You could visit the Eiffel Tower, the Louvre Museum, or Notre-Dame Cathedral."

Creating the Conversation Dataset

Supervised fine-tuning relies on high-quality conversational datasets crafted by human annotators following clear guidelines.

Dataset Contains

01

User Questions

Natural language prompts representing real user queries

02

Ideal Assistant Responses

Carefully crafted answers that are helpful, accurate, and safe

Annotator Guidelines

Helpful

Accurate

Safe

Conversation Structure

User:

Question

Assistant:

Response

This structure teaches the model:

1

Dialogue Flow

How conversations naturally progress

2

Multi-Turn Conversations

Handling sequences of exchanges

3

Context Awareness

Maintaining context across messages

Training the Model with SFT

During supervised fine-tuning, the model's parameters are updated so that it learns to generate responses similar to curated human examples.



Training via Backpropagation

Billions of model parameters are gradually adjusted through backpropagation to improve:

- 01 Natural Language Flow**
Smooth, human-like sentence construction and transitions
- 02 Context Awareness**
Understanding and responding to the full conversation history
- 03 Response Accuracy**
Generating factually correct and relevant information

Result

The model starts behaving more like a conversational assistant rather than a pure text predictor.

Before: "Paris is a city in France that..."

After: "Sure! Here are great spots..."

Personality & Multi-Turn Dialogue

Supervised fine-tuning also allows models to develop a consistent conversational style and handle extended interactions.

Conversational Tones

Depending on training data, the model can adopt different styles:

Friendly & Approachable

"Hey! Great question — let me help!"

Professional & Formal

"The recommended approach is as follows."

Informative & Instructional

"Step 1: Open the terminal. Step 2: Run..."

Multi-Turn Capabilities

Training with multi-turn dialogue examples helps the model learn to:

01

Reference Earlier Messages

Connecting current response to previous exchanges

02

Maintain Context

Keeping track of topics and entities across turns

03

Coherent Over Multiple Turns

Avoiding contradictions and staying on topic

Result

Interactions feel more natural, engaging, and contextually aware.

Reinforcement Learning with Human Feedback

After SFT, many models undergo RLHF to further align responses with human preferences. Unlike SFT which teaches by examples, RLHF teaches by ranking responses.

RLHF Workflow

01

Generate Candidates

Model produces multiple candidate responses for a given prompt

02

Human Ranking

Humans compare and rank responses from best to worst

03

Train Reward Model

A reward model learns these human preferences as a scoring function

04

Optimize Model

Model is optimized to generate responses that receive higher rewards

RLHF Improves

Response Relevance

Answers are more directly useful and on-topic

Tone & Politeness

Appropriate, respectful communication style

Safety & Alignment

Responses aligned with user expectations and guidelines

Challenges & New Approaches

While RLHF significantly improves model behaviour, it introduces several challenges that have driven the development of alternative methods.

RLHF Challenges



Reward Hacking

Model may exploit the reward system without truly improving answer quality



High Cost

Collecting human feedback at scale is expensive and time-intensive



Balancing Creativity & Accuracy

Difficult to optimise for both simultaneously without trade-offs

Direct Preference Optimization

A Newer Alternative

DPO simplifies training by:



Directly optimising the model using human preference comparisons



Avoiding the need for a separate reward model



Reducing training complexity and compute cost

Key Takeaway

Many modern systems combine Supervised Fine-Tuning, RLHF, and DPO to produce high-quality conversational AI models.

Applications & Capabilities

Common Real-World Use Cases of LLMs Across Various Domains

Large language models power practical systems across industries by enabling natural language understanding, generation, and reasoning.

01 Conversational Agents

Virtual assistants, chatbots, and multi-turn dialogue systems for customer support and enterprise workflows.

02 Content Creation

Drafting articles, summarisation, translation, grammar correction, and marketing copy generation.

03 Coding Support

Code generation, autocompletion, debugging, refactoring, and test case creation for developers.

04 Knowledge & QA

Question answering, structured explanations, research assistance, and intelligent knowledge retrieval.

Key Insight

These capabilities make LLMs foundational infrastructure for intelligent software systems across every industry.

Conversational Agents & Content Generation

Conversational Systems

- 1 Chatbots for customer support
- 2 AI assistants for enterprise workflows
- 3 Multi-turn contextual dialogue systems
- 4 Automated service agents in applications

Maintain context across interactions and generate human-like responses.

Text Summarisation

- 1 Condensing long reports into short insights
- 2 Extracting key points from research papers
- 3 Creating executive summaries from documents

Content & Writing Assistance

- 1 Drafting blogs, articles, and technical docs
- 2 Grammar correction and rewriting
- 3 Marketing content generation

Language Translation

- 1 Real-time multilingual translation
- 2 Context-aware translation preserving meaning
- 3 Cross-border communication support

Coding Assistance & Question Answering

Coding Assistance

LLMs significantly improve developer productivity.

GitHub Copilot

AI code autocompletion and contextual coding support

Claude Code

AI-assisted coding with project-level understanding

Capabilities:

- 1 Code generation from comments
- 2 Bug detection and debugging
- 3 Refactoring suggestions
- 4 Test case generation

Question Answering

LLMs act as knowledge-based systems trained on large datasets.

Example Queries

- "Why is the sky blue?"
- "Explain quantum computing in simple terms."
- "How does `async/await` work in Python?"

How It Works

The model retrieves patterns from its training data and generates structured explanations — functioning like an intelligent knowledge assistant.

Limitations & Reasoning Challenges

Despite strong performance, Large Language Models face fundamental constraints that impact reliability and accuracy.

Challenge

01 Complex Reasoning

Struggles with multi-step logical and mathematical problems.

- Multi-variable logic
- Long dependency reasoning
- Symbolic computation

Example

12×15 may produce errors from pattern matching alone

Challenge

02 Context Window

Limited memory for long conversations and documents.

- Earlier info lost beyond capacity
- Reduced long-form accuracy
- Token limit constraints

Challenge

03 Hallucination

Confident but incorrect or fabricated outputs.

- Fabricated references or facts
- Inconsistent responses
- Probabilistic generation variance

Mitigation Strategies

Reasoning methods and tool integration help overcome core LLM limitations and improve reliability.

Chain-of-Thought Reasoning

Models generate intermediate reasoning steps before producing the final answer.

Benefits

- ✓ Improves transparency
- ✓ Enhances structured problem solving
- ✓ Reduces logical errors

Trade-offs

- Increased token usage
- Possible reasoning flow interruption

Tool Integration

Connecting LLMs to external tools improves reliability and precision.

Calculator

Accurate arithmetic

Search Engines

Up-to-date information

Databases

Real-time data access

Code Execution

Runtime validation

Tool Usage Workflow



Enables grounded reasoning and reduces dependency on internal knowledge alone.

Challenges & Risks

AI systems require continuous monitoring, defensive safeguards, and governance mechanisms to reduce risk while maintaining capability.

Security Risks

Prompt Injection

Manipulating instructions to override safeguards

Jailbreaking

Bypassing safety constraints

Data Poisoning

Injecting harmful data into training

Backdoor Attacks

Hidden triggers to alter behaviour

Ethical Considerations

Training Data Bias

Outputs affected by biased training data

Misinformation

Hallucinated content spread as fact

Privacy Risks

Sensitive data processed without controls

Environmental Impact

Large-scale training and inference costs

Mitigation Approaches

- ✓ Human oversight and evaluation
- ✓ Robust alignment training
- ✓ Red-team testing for vulnerabilities
- ✓ Continuous model updates
- ✓ Policy controls and usage restrictions

Key Takeaway

No single solution solves all risks. Modern systems combine technical safeguards with governance mechanisms to reduce risk while maintaining capability.

Multimodal AI

Processing text, images, audio, and video in a unified model

Multimodal AI combines multiple data types into a single model capable of reasoning across all of them simultaneously — unlike earlier systems limited to one modality.



Leading Players

Google Gemini 3.1 Pro	Text, image, audio, video + 1M token context
OpenAI GPT-5	Native voice, vision & real-time interaction
Anthropic Claude Opus 4.6	Document, image & code understanding
Meta Llama 4	Open-weight multimodal, 10M token context

Shaping the Future

- 1 50% of consumers now prefer multimodal interactions as their primary format
- 2 Agents can analyse videos, PDFs, code, and speech in a single session
- 3 Healthcare, retail, and education are being transformed by unified perception models

Autonomous AI Agents

AI that plans, executes, and iterates across multi-step workflows with minimal human input

40%

Enterprise apps with
AI agents by end 2026

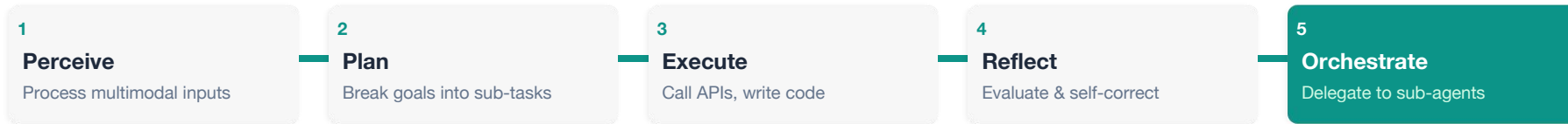
\$52B

Projected agentic AI
market by 2030

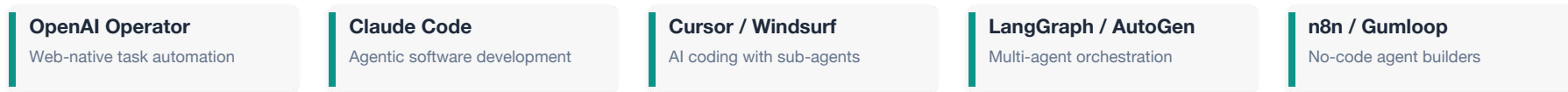
90%

Cost reduction with
Plan-and-Execute agents

How They Work



Key Platforms

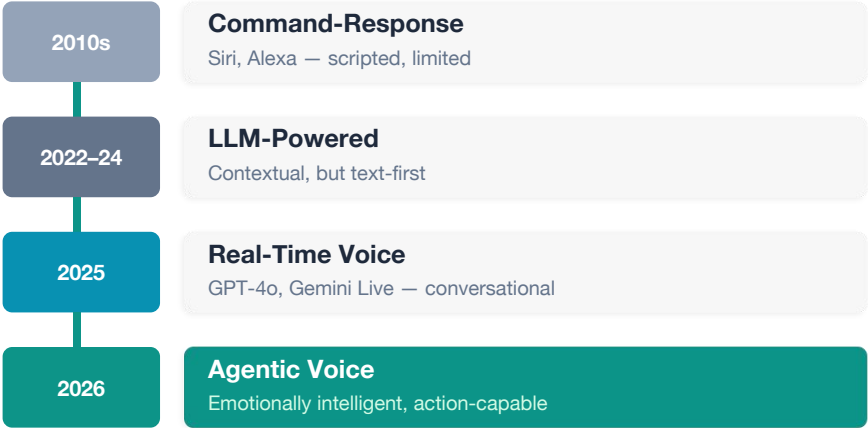


By 2027, 70% of multi-agent systems will use narrow, specialised agents rather than generalist ones — improving accuracy significantly.

Voice-Based AI Systems

Natural, bidirectional voice AI with emotional intelligence, multilingual support, and agentic capabilities

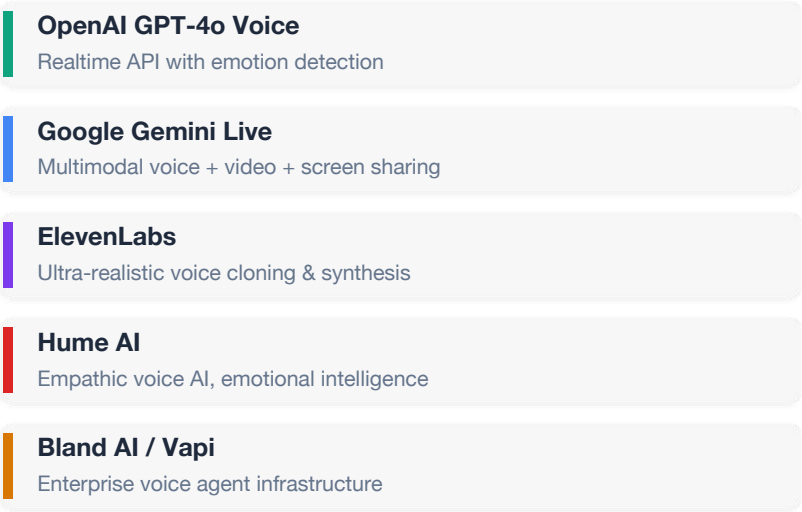
The Evolution of Voice AI



2026 Capabilities



Leading Platforms



Emotional AI market: \$19.5B (2020) → \$37.1B (2026)

Physical AI & Embodied Intelligence

AI systems that perceive, reason, and act in the physical world

Humanoid Robots

Figure 02, Tesla Optimus, Unitree G1 & H2 — general-purpose robots learning from video demonstrations and LLM-based planning. Moving toward household and industrial task execution.

Autonomous Vehicles

Waymo, Tesla FSD, Zoox — navigating unstructured environments using real-time multimodal sensor fusion. Waymo operates fully driverless commercial fleets in multiple US cities.

Industrial Robotics

Boston Dynamics Spot, Amazon Proteus — warehouse, logistics, and factory automation with adaptive manipulation. AI handles variability that previously required hard-coded programming.

Drone Swarms & UAVs

Zipline, Shield AI — autonomous aerial agents for medical delivery, surveillance, and emergency response. Shield AI's Hivemind enables coordinated multi-drone missions.

Key Signal

AI is escaping the screen — the physical world is becoming the next frontier for autonomous intelligence.

Frontier AI Models — 2026

No single model dominates everything — the current generation shows genuine specialisation

Gemini 3.1 Pro

Google DeepMind

Best for: Abstract reasoning, multimodal

- #1 ARC-AGI-2 (77.1%)
- 1M token context window
- \$2/\$12 per 1M tokens

Claude Opus 4.6

Anthropic

Best for: Agentic workflows, code review

- #1 SWE-Bench Verified (80.8%)
- #1 Agentic Elo (1,606)
- HLE with tools: 53.1%

GPT-5.2

OpenAI

Best for: Software eng, enterprise

- Leader on SWE-Bench Pro
- GPQA Diamond: 92.4%
- Strong multimodal support

Qwen 3.5

Alibaba

Best for: Open-source, multilingual

- 397B MoE architecture
- Open-weight, self-hostable
- Leading multilingual scores

Key Benchmarks & Performance

Source: February–March 2026 data

Benchmark	What It Tests	Gemini 3.1 Pro	Claude Opus 4.6	GPT-5.2 Pro
ARC-AGI-2	Novel abstract reasoning	77.1%	68.8%	54.2%
HLE (no tools)	Expert-level knowledge	44.4%	40.0%	36.6%
HLE (with tools)	Tool-augmented reasoning	51.4%	53.0%	50.0%
GPQA Diamond	Graduate-level science	94.3%	91.3%	93.2%
SWE-Bench Verified	Real-world code fixes	80.6 %	80.8%	80.0%
SWE-Bench Pro	Complex software eng.	54.2%	—	55.6%
Agentic Elo	Expert-level office work	1,317	1,606	1,462

2026 marks the end of one-model dominance. Route by strength — Gemini 3.1 Pro for abstract reasoning & science (ARC-AGI-2, GPQA), Claude Opus 4.6 for agentic & knowledge work (HLE-tools, SWE-Verified, GDPval-AA), GPT-5.2 Pro for engineering depth (SWE-Pro).

Companies Leading the AI Space

A three-tier ecosystem around frontier AI infrastructure, enterprise deployment, and agent-native startups

Tier 1 — Frontier Model Labs

Anthropic

Claude 4 family; leads agentic workflows & safety

OpenAI

GPT-5 series; dominant adoption

Google DeepMind

Gemini 3.x; benchmark crown; unmatched multimodal

xAI (Grok)

Grok 4.x; parallel multi-agent architecture

Tier 2 — Open-Weight

Meta AI (Llama 4)

10M token context; fuels open-source ecosystem

Alibaba (Qwen 3.5)

397B MoE; leading open-weight multilingual

DeepSeek

Frontier-class at \$0.28/1M tokens

Mistral AI

Efficient MoE; strong EU open-source presence

Tier 3 — Agent-Native

Cursor / Windsurf

AI-first coding IDEs with multi-agent execution

ElevenLabs

Leading voice synthesis & cloning platform

Vellum / Gumloop / n8n

Agentic workflow automation platforms

Hume AI

Empathic voice AI; emotion-aware systems